

Inigo Quilez :: articles :: simple water - 2004



Intro

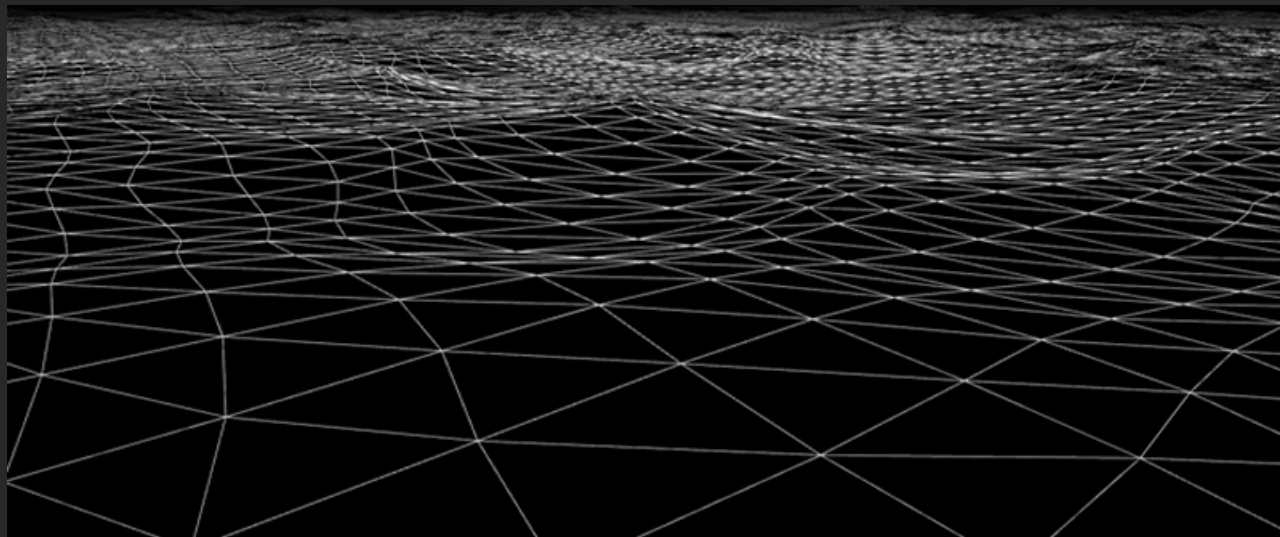
Water is one of those things you try first when you learn shading. It involves some math calculations, picking some colors, thinking a bit physically, cheating reflections, doing some fresnel effects, and basically everything you do in most other shaders. Everybody does water differently, and this short article contains only one implementation possible. In fact, this is the recipe for the water shader of the *Paradise* 64 kilobyte demo done in 2004 (with ARB shaders, before GLSL was here, so I'll give explanations instead of ugly assembler-like code):



*Screen capture from the realtime demo *Paradise**

[1] We will be using a regular triangle mesh for the water surface. Probably this is a regular grid which has been distorted by moving the vertices vertically with some perlin noise variation. This should be animated with time (done in the CPU or in a vertex shader) to simulate waver moving with wind. Additionally to the vertical

displacement, you most probably want to move the vertices **horizontally** too, which most people forget to do. In *Paradise*, I moved them proportionally to the height of the wave, so that the surface got more choppy. That helped me to make the desert surface too.



Wireframe view of the mesh. The horizontal vertex displacement gives a nice sharp profile to the waves.

[2] Compute some animated fbm texture. Four octaves is probably enough in most cases. This can be done with a 3D perlin noise function or even faster with a series of simple perlin noise octave textuers which are moved at different speeds and added together to make the final animated fbm. The article in [cloud rendering](#) explains how to do this. We will be using this dynamic fbm texture as normal map to give extra detail to the water surface.



*The horizontal displacement can be used to simulate deserts too (from the *Paradise* demo).*

[3] Compute or draw a cubemap representing the sky and distant objects like far away islands or coast. If

performance is an issue, compute this once or draw it in your favourite image editing program. If you want higher quality and accuracy, compute this cubemap every frame of the demo by rendering the environment six times with 90 degree frustums.

[4] Get ready another texture to fake the reflections on the water. You can do this by rendering the objects you want to be reflected in the water flipped vertically. This includes the icebergs, ships, the coastline etc. You will need to set a clipping plane so everything below the water level is not rendered into this texture. After you got this texture, it would be a good idea to blur it a bit, so that later reflection are not sharp but soft. Also, you don't need to render in full resolution at this step, a 2x2 times smaller texture would do it too.



Reflection texture (blurred)

[5] We will need another texture, the refraction texture, that will contain those parts of the object that are below the water level but are still partially visible. Usually objects become non visible gradually as they sink. So, in this texture we will be painting the coast, deck of ships and bottom part of icebergs. We will need another clipping plane to avoid rendering everything above the water level. But not only we need the refracted color in this texture, we also need the zbuffer value of the image at every pixel, as this will allow us to correctly fade objects away later as they are deeper in the water.

[6] Then we are ready to start with the shader. First thing will be to distort the surface normal with the normal map. You can read how to do this in any tutorial on "normal mapping".

[7] We will be computing some regular diffuse lighting with the usual $N \cdot L$ formula.



Partial water shader, under good weather conditions (2005).

[8] Then we can modulate the base water color with the diffuse component. This color is a linear blend of blue and green. The blending factor will be based in the $N \cdot E$ quantity, where E is the eye vector, so that flat parts of the water get blue and the wave sides are greenish. This simulates the subsurface scattering of light across the wave.

[9] We will be computing some fresnel term by taking some power of $N \cdot E$. We will add some white color to the water color we had so far modulated by this fresnel. We will also add some reflection by fetching pixels from the cubemap according to the reflected vector R - like in $\text{reflect}(N, E)$ - at the shading point, also modulated by the fresnel term.

[10] If the sun is not in the reflection cubemap (it shouldn't), then we will add also its reflection with a regular specular computation (based on R too).

[11] We will also add the reflection of the close objects by fetching from the reflection map/texture we computed before. We want to add this color by first modulating it with some sort of reflection coefficient. Probably we want to reflect more or less based in the angle between E and N . The sampling position should be the same pixel we are shading offset by some little amount dependant on the normal. That way, reflections will be no perfect mirror but will contain ripples.

[12] Now, we add the refraction color which accounts for the underwater visible objects. By reading the reflection image we first get the zbuffer value at the pixel for the refracted object. We compare this to the

current zbuffer value, which is the one of the water surface. By subtracting one from the other we know how "deep" the refracted object is in the water. Of course, before fetching that pixel's zbuffer we offset the sampling point a bit based again in the angle between N and E, so simulate the actual refraction. When we know the depth of the object, we use the weight to both lerp between the color of the object and black (no refraction) and to actually change the tint of the object toward green or blue. That will make objects underwater but close to the surface look greenish, and those deeper down be bluish.

[13] Lastly, we put some foam in the top of the waves by adding some random white color fetched from a "foam texture" based on the height and slope of the wave mesh. We can add a time offset to the fetching coordinates that is aligned to the wind direction, to let the foam "slide" a bit over the water surface.

[14] There is a last thing you can do if you want to integrate the ocean surface with a weather simulation effect, and that's to add rain drop splashes to the water surface by adding a texture with random white dots that is regenerated every frame. It will of course not match the actual collision points of the rain drops with the water, but nobody will notice of course.



Partial water shader, under bad weather conditions (2005).

There are many constants, powers, factors and colors involved here, so it's a bit of an art to get the right ones. You probably want to play with the shader for a couple of days before you are completely happy.

Also, as there are many effects involved in this shader (several reflections, refractions, color blends, fresnels, speculars, diffuse, raindrops, foam, etc) you can decide how many of them you want to implement and how many you prefer to leave out to save some framerate, so basically you can do water in any platform, it's just about how many features you add to it.

inigo quilez - learning computer graphics since 1994